

Information Retrieval: Einfache IR-Modelle und ihre Implementierung

Ingo Schmitt

Brandenburgische Universität Cottbus

Institut für Informatik

Lehrstuhl für Datenbank- und Informationssysteme



Gliederung

4.1 Pattern-Matching

4.2 Wortsuche

4.3 Boolesches Retrieval

4.4 Coordination Level Match

4.5 Extended Boolean

4.6 Fuzzy-Mengen-Modell

IR-Modell

implizite wie auch explizite Aspekte:

- Annahmen

- etwa: egal, ob Wort einmal oder zehnmal im Dokument vorkommt
- Unabhängigkeit der einzelnen Indexworte

- Repräsentation von Dokumenten und Anfragen

- Retrieval-Funktion

- Berechnung Relevanzkennzahl für ein Paar aus Anfrage und Dokument
- Näherung der Relevanzwahrscheinlichkeit

4.1 Pattern-Matching

- Sehr einfaches IR-Modell
- in einigen IR-Modellen als Teilmodell
- Aufgabenstellung: Suche einer Zeichenkette (Such-Pattern, Suchmuster) in längerem Text
- Realisierung:
 - Suchmuster der Länge m von links nach rechts sequenziell mit dem zu durchsuchenden Text der Länge n vergleichen
 $\Rightarrow O(m*n)$
 - Bessere Algorithmen: Boyer-Moore-Horspool-Sunday-Algorithmus, Knuth-Morris-Pratt-Algorithmus
 $\Rightarrow O(n)$

4.2 Wortsuche

- Sehr einfaches IR-Modell
- Aufgabenstellung:
 - Gegeben: Wort W
 - Gesucht: alle Dokumente, die W enthalten
- mögliche Erweiterung: Verwendung Stamm- oder Grundformreduktion

Wortmenge

Ausweitung auf Wortmenge $\{W_1, \dots, W_k\}$

- Suche nach Dokumenten, die alle gegebenen Worte enthalten
- implizite UND-Verknüpfung über die gesuchten Worte
- erster Schritt zum Booleschen Retrieval

Unterschied zum Pattern-Matching

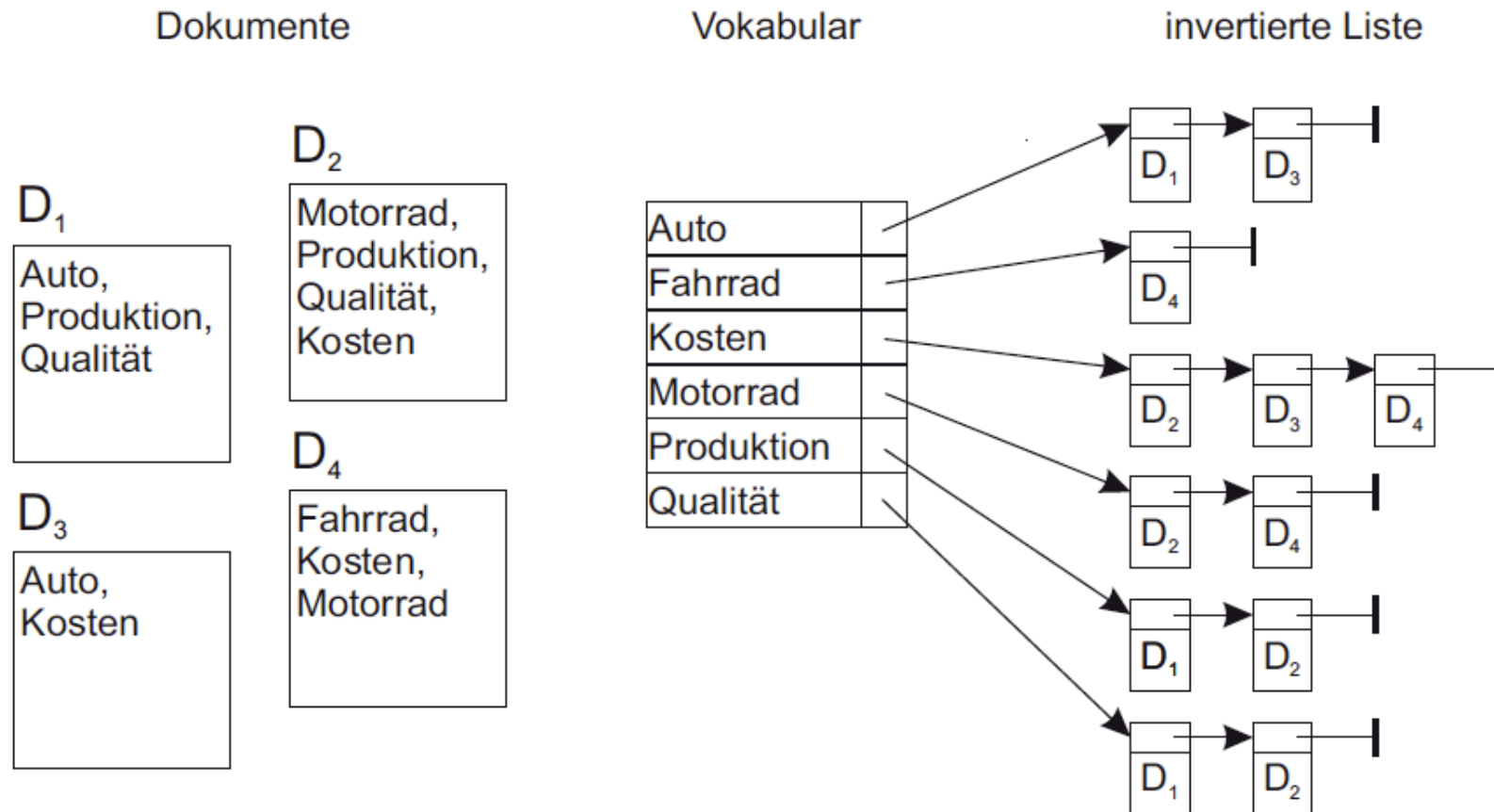
- Kein Test, ob gegebenes Pattern bzw. Suchwort in **einem** Text enthalten ist, sondern Ermittlung **Menge** von Suchwort enthaltenden Dokumenten
⇒ Erweiterung zu Wortmengen möglich
- Suchwort muss im Dokument als eigenständiges Wort auftreten

Implementierungsansätze für die Wortsuche

- Pattern-Matching-Algorithmus auf jedem Dokument und anschließender Test auf eigenständiges Wort
 - Problem: Suchaufwand wächst linear mit Umfang der Dokumentkollektion
- Indexstruktur vorab erstellen
 - Invertierte Listen
 - Signaturen

Invertierte Listen

zu jedem Wort wird eine sortierte Liste der Dokumente verwaltet,
welche dieses Wort enthalten



Wortsuche: Suchwort

Suche nach allen Dokumenten, die exakt das Suchwort enthalten

- Durchlauf der invertierten Liste zum vorgegebenen Suchwort

Wortsuche: Gleicher Wortstamm

- Suche nach Vorkommen mit gleichen Wortstamm:
 1. Wortstamm zu Suchwort bilden
 2. Temporäre Listenmenge erstellen: alle Listen zu Wörtern, deren Wortstamm identisch zum Wortstamm des Suchworts ist
 3. Vereinigungsmenge der Dokumenten-Ids der Listen der Listenmenge bilden
 4. Dokumente der Vereinigungsmenge bilden Ergebnis der Anfrage

Wortsuche: Gleicher Wortstamm

- Effiziente Bestimmung der Vereinigungsmenge über mehrere Listen mittels *Sortierung der Listen nach Dokument-Ids*
 - Listen werden parallel durchlaufen
 - Ausnutzung Sortierung:
 1. Initialisieren aller Listen auf jeweils erstes Element
 2. kleinstes aktuelles Element kommt in Ergebnisliste
 3. Listen mit kleinstem aktuellem Element werden auf nächstes Element gesetzt
 4. wenn insgesamt noch Elemente vorhanden sind
⇒ Schritt 2

Wortsuche: nach Wortmenge

- Schnittmengenbildung über betroffene Listen
- Listen werden parallel durchlaufen
- Ausnutzung der Sortierung der Listen
 1. Initialisieren aller Listen auf jeweils erstes Element
 2. Alle aktuellen Elemente gleich
⇒ Übernahme in Ergebnisliste
 3. Alle aktuellen Elemente nicht gleich
⇒ Listen mit kleinstem Element auf nächstes Element
 4. wenn insgesamt noch Elemente vorhanden sind
⇒ Schritt 2

Implementierung invertierter Listen

Erforderlicher Speicherplatz:

■ Beispiel TREC-Kollektion

- enthält Artikel aus *Financial Times*
- Umfang der Dokumentenkollektion beträgt 564 MByte
- Kollektion enthält 210.158 Artikel, die im Mittel ca. jeweils 150 unterschiedliche Wörter haben
- Annahme: 4 Bytes pro Dokument-ID
 $\Rightarrow 210.158 \cdot 150 \cdot 4 \text{ Byte} = 126.094.800 \text{ Byte} = 120 \text{ MByte}$
- Unberücksichtigt: Begriffe selbst, Verwaltungsdaten, Vorkommenshäufigkeit, -ort
- Fazit: Index kann schnell 20% der Dokumentkollektion erreichen $\Rightarrow$ Speicherung auf Hintergrundspeicher

Zipf's Law

Beobachtung beim Betrachten von Texten:

- einige wenige Wörter kommen sehr häufig vor
- englische Texte:
 - die beiden am häufigsten vorkommenden Wörter stellen 10 % aller Vorkommen
 - die sechs häufigsten Wörter stellen 20 % aller Vorkommen
 - die fünfzig häufigsten Wörter stellen 50 % aller Vorkommen
- Sehr viele Worte kommen sehr selten vor

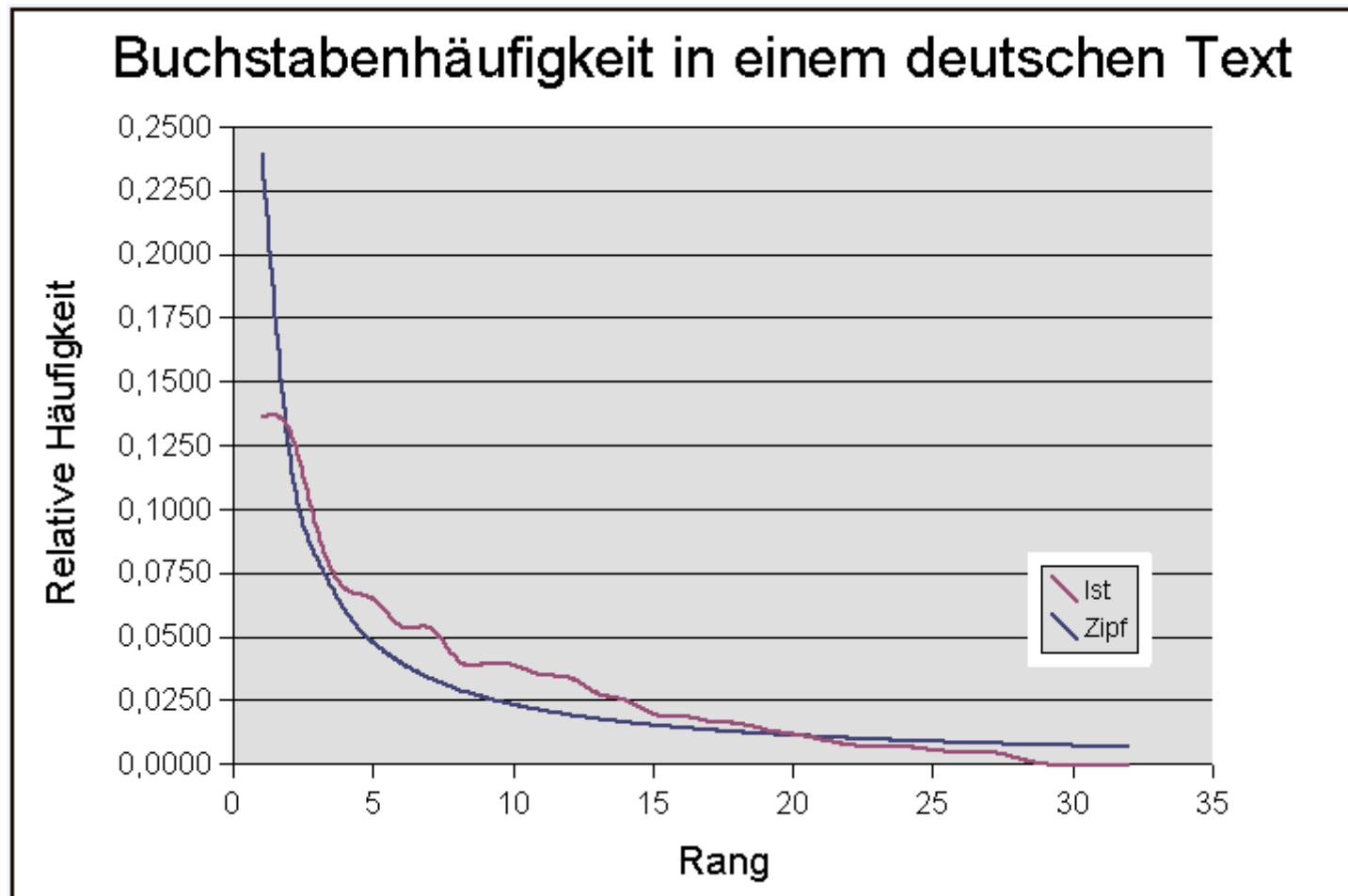
Zipf's Law: Brown und Lob

Rang $r(w)$	Anzahl $h(w)$	$\frac{r(w) \cdot h(w)}{100.000}$	Wort bzw. Term
1	138.323	1,38323	the
2	72.159	1,4432	of
3	56.750	1,7025	and
4	52.941	2,1176	to
5	46.523	2,3262	a
6	42.603	2,5562	in
7	22.177	1,5524	that
...	...	...	...
2804	73	2,0476	destroy
2805	73	2,0476	determination
...	...	...	...
12032	11	1,3235	would-be
12033	11	1,3236	yachting
12034	11	1,3237	yell

Zipf's Law

- Sortierung der Wörter nach Vorkommenshäufigkeit
⇒ Produkt aus Rangzahl und Vorkommenshäufigkeit ist konstant (Zipf's Law)
 - Häufigste Wort ca. 7 Millionen oft
 - Zweithäufigste Wort ca. 3,5 Millionen oft
 - Dritthäufigste Wort ca. 2,3 Millionen oft
- Formal:
 - repräsentativer Textcorpus C
 - $W(C)$ ist Menge der Wörter, die in C vorkommen
 - $h(w)$ ist Häufigkeit, mit der ein Wort $w \in W(C)$ im Korpus vorkommt
 - $r(w)$ bezeichne den Rangplatz von $w \in W(C)$, wenn die Wörter nach abfallender Häufigkeit sortiert werden
- Dann gilt: $r(w) \cdot h(w) \approx c$ (konstant), $\forall w \in W(C)$

Zipf's Law



Beispiel aus Wikipedia (27.05.2009)

Zipf's Law und invertierte Listen

Schlussfolgerung:

- Wenige lange Listen
trotz Entfernung von Stoppwörtern
- Viele kurze Listen
trotz Entfernung sehr seltener Wörter (Crouch)
- Suchstruktur zur Abbildung Wörter auf entspr. Liste nötig
⇒ z.B. B-Baum oder Digitalbäume

Exkurs: B-Baum

- Bekannt aus VL Datenbanktechnologie
 - Anwendung in Datenbanken und Dateisystemen
- Balancierter Baum
- Speicherung der Daten sortiert nach Schlüsseln
- Blattknoten mit Daten

Verwaltung invertierter Listen

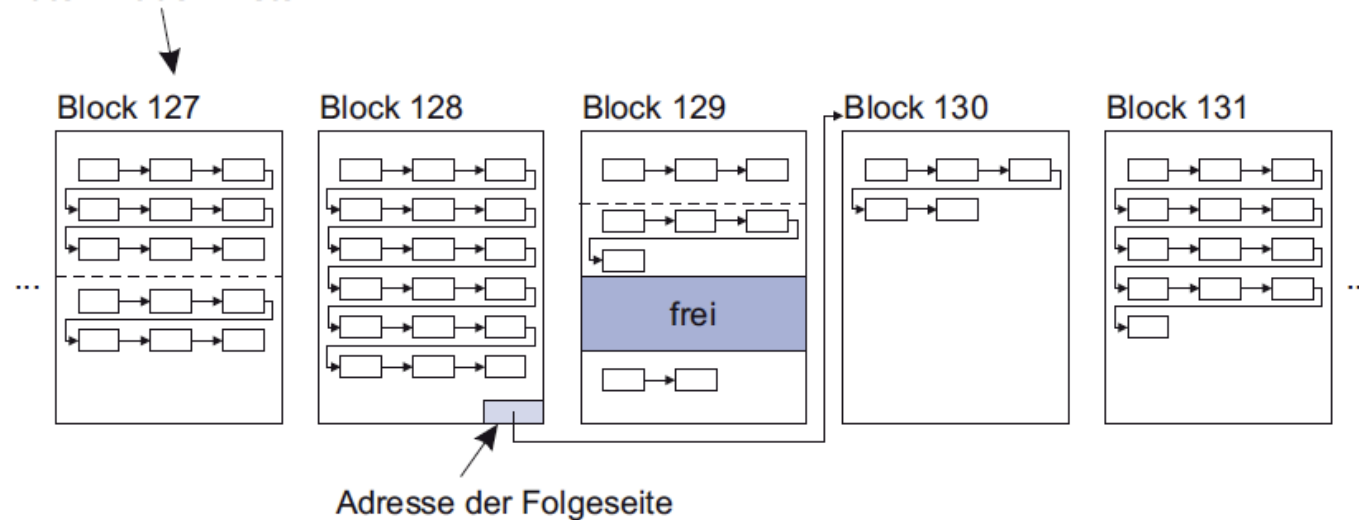
- Persistente Verwaltung auf Hintergrundspeicher
- Verwaltung in Blöcken/Seiten
 - 1:1-Zuordnung zwischen Listen und Seiten wegen Variabilität der Listenlängen nicht sinnvoll
 - Mehrere kleine Listen auf eine Seite
 - Große Listen über mehrere Seiten hinweg
- Mehrstufige Adressierung einer Liste
 1. Dateikennung
 2. Blocknummer in Datei
 3. Anfangsadresse im Block (zusätzlich auch Länge)

Verwaltung invertierter Listen

Verwaltungstabelle

Term	Blocknum.	Länge	Startposition
...	...	...	...
Goldfisch	129	256	0
Hund	128	1024	0
Katze	127	512	512
Kuh	131	1024	0
...	...	...	...
Schlange	129	256	768
Tiger	127	512	0
...	...	...	...
Zebra	129	256	256

Datei mit den Listen



Signaturen

- Alternative zu invertierten Listen
- Hash-Abbildung l-stelliges Wort $w_1w_2..w_l$ auf **Signatur** (Bitstring fester Länge F)
- Effiziente Vergleichsoperationen auf Signaturen
- Beispiel Hash-Abbildung:

```
hash := 0;  
for i := 1 to l do  
    hash := (hash +  $w_i$ ) * 157;  
end  
hash := hash mod  $2^F$ ;
```

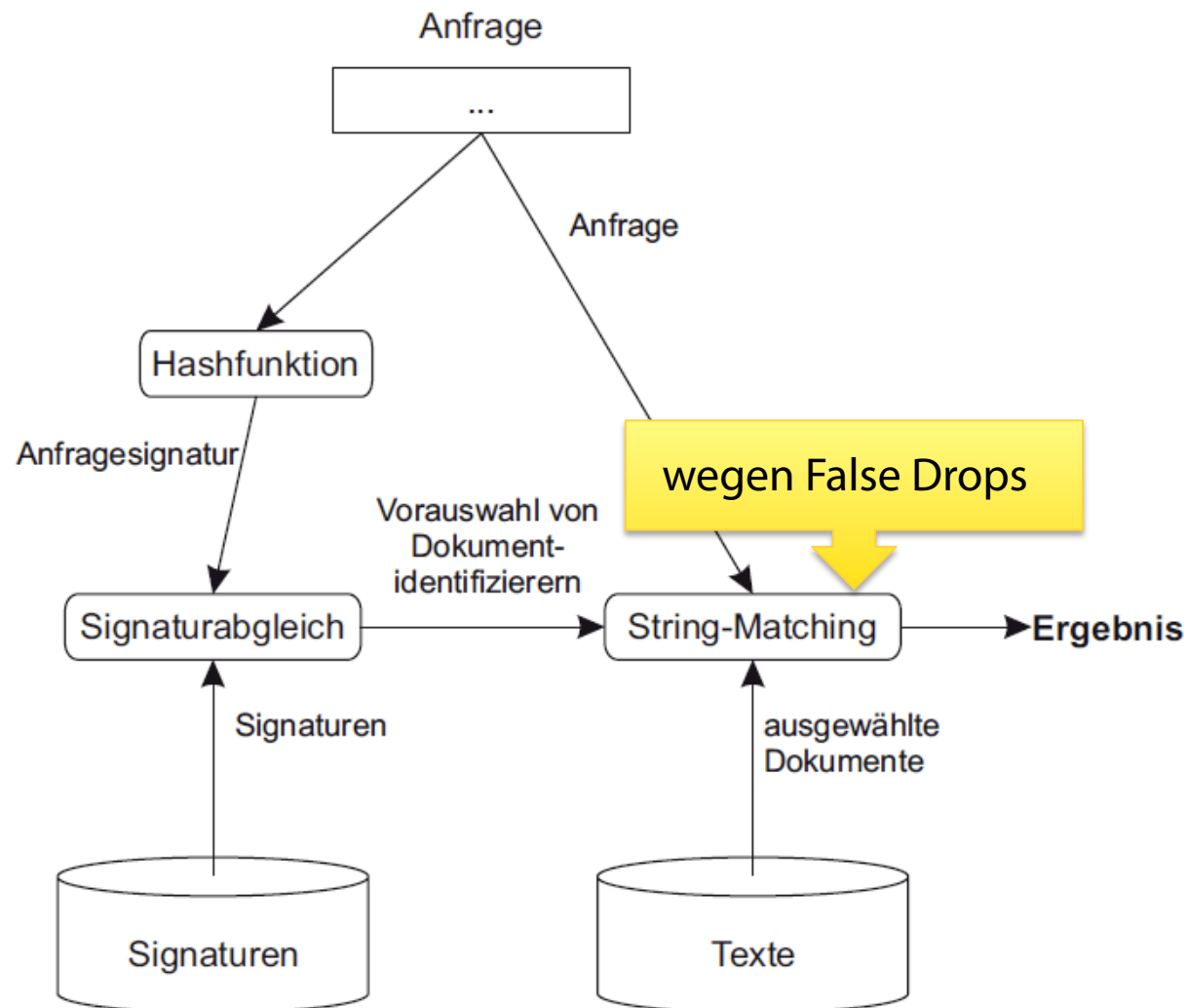
Eigenschaften von Signaturen

- Hashfunktion h bildet üblicherweise großen Urbildraum auf kleinen Bildraum ab (Filter)
- $h(wort_1)=h(wort_2)$ mit $wort_1 \neq wort_2$ (Kollision) möglich, d.h. unterschiedliche Worte erhalten gleichen Hash-Wert
- Operationen auf Signaturen daher nicht exakt

Idee der Signaturdatei

- Signaturdatei als **inexakter Filter**
- Überprüfung notwendiger Bedingung, damit Dokument Wort enthält
- Inexakt $\Rightarrow$ trotz Erfüllung der Bedingung muss Dokument Wort nicht erhalten (**false drop**)
- False drops müssen im Nachhinein herausgefiltert werden
- Aber: Nachfilter auf relativ wenigen Dokumenten

Signaturdatei: Architektur



Motivation für Signaturen

- Pattern-Matching
 - Alle Dokumente müssen **immer** durchlaufen werden
- Signaturen
 - Signaturvergleich **schneller** als Pattern-Matching
 - Kandidatenauswahl
 - Pattern-Matching auf **kleinerer** Menge von Dokumenten

Überlagernde Signaturen

Superimposed Coding:

- Abbildung 1 Wort auf 1 Signatur bestehend aus F Bits mit m (Signaturgewicht) gesetzten Bits
 $\Rightarrow \binom{F}{m}$ verschiedene Signaturen möglich
- Signaturen mehrerer Wörter werden bitweise ODER-verknüpft $\Rightarrow$ **Blocksignatur**
- Weniger Signaturen pro Dokument

Blocksignatur: Beispiel

$F=12$, $m=4$

Wort	Signatur
free	001 000 110 010
text	000 010 111 000
Blocksignatur	001 010 111 010

Hash-Funktion

Problem: Hash-Wert soll pro Wort m gesetzte Bits im Bitcode der Länge F erzeugen

- Verwendung von mehr als m Hash-Funktionen mit Abbildung auf $[1..F]$
- Ergebnisse geben Position der gesetzten Bits an
- Gleiche Positionen möglich
⇒ Nutzung der m ersten **verschiedenen** Positionen

Hash-Funktion

- i-te Hash-Funktion mit Abbildung auf $[0..F-1]$
- Argument: l-stelliges Wort $w_1w_2..w_l$
- i-te Primzahl p_i

```
hashi  
for k := 1 to l do  
    hashi := (hashi + wk) * pi;  
end  
hashi := hashi mod F;
```

Parameter für Superimposed Coding

- Signaturlänge F
- Signaturgewicht m
- Überlagerungsfaktor D :
Anzahl Wortsignaturen pro Blocksignatur

Logische Blöcke

- Dokumente werden in logische Blöcke zerlegt
- Ein logischer Block enthält genau D verschiedene zu indexierende Wörter
- jeder logische Block eine Blocksignatur
- Ergebnis: mehrere Blocksignaturen für ein Dokument

Suche nach Dokumenten

Suche nach Dokumenten, die Wort w enthalten

- Berechnung Wortsignatur S_w für Wort w
- Bitweise UND-Verknüpfung von S_w mit jeder Blocksignatur S_{bi} der Signaturdatei
- Falls $S_w = S_w \wedge S_{bi}$, dann entsprechendes Dokument in Kandidatenmenge
- Entfernen von False Drops aus Kandidatenmenge mittels Pattern-Matching

Konjunktive Anfrage

- mehrere mit UND verknüpfte Suchwörter
- $\Rightarrow$ ODER-Verknüpfung der Signaturen der Suchwörter der Anfrage
- $\Rightarrow$ nur Dokumente gefunden, bei denen alle Anfragewörter **innerhalb eines Blocks** liegen

Block	Suchwort	Signatur
1	Information	001
2	Retrieval	100

- Anfrage „Information Retrieval“ (101) würde das Dokument nicht finden

Konjunktive Anfrage

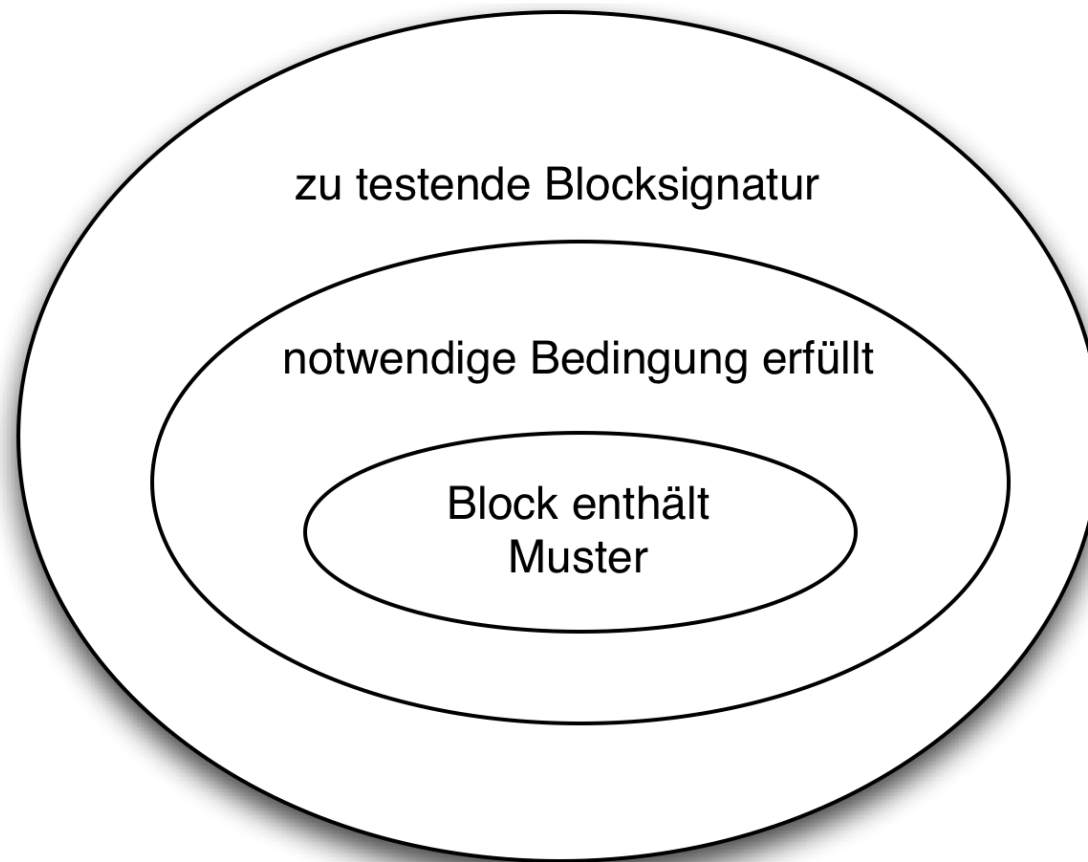
- Alternative:
 - für jedes Anfragewort Kandidatenmenge bestimmen
 - Vor Pattern-Matching $\Rightarrow$ Berechnung der Schnittmenge
- Vorteil: Verfahren funktioniert immer
 - Wie geht man bei ODER-Verknüpfung vor?

False Drops

- können bei Signaturdateien auftreten
- dies bedeutet:
 - Block bzw. dessen Blocksignatur erfüllt notwendige Bedingung
 - das zu suchende Muster muss aber nicht im Block bzw. dem dadurch repräsentierten Dokument enthalten sein
- Entstehung:
 - Hash-Funktion kann verschiedene Wörter auf identische Signatur abbilden
 - durch Überlagerung der Signaturen

False Drops

False Drops = notwendige Bedingung erfüllt – Block enthält Muster



False Drops: Beispiel

■ Beispiel:

■ Es gelte: $F = 12$, $m = 3$ und $D = 3$

■ Wortsignaturen:

■ Anfrage „text“ UND „search“

Wort	Signatur
text	010 010 001 000
search	010 000 100 100
method	100 100 000 001
full	010 110 000 000
knowledge	000 111 000 000
retrieval	000 000 111 000
lexicon	100 010 010 000

False Drops: Beispiel

Wort	Signatur
text	010 010 001 000
search	010 000 100 100
method	100 100 000 001
full	010 110 000 000
knowledge	000 111 000 000
retrieval	000 000 111 000
lexicon	100 010 010 000

Textblöcke	Blocksignaturen
text search method	110 110 101 101
search knowledge retrieval	010 111 111 100
full text search	010 110 101 100
lexicon knowledge retrieval	100 111 111 000

Kandidaten sind die ersten 3 Textblöcke

2. Block ist False Drop

Minimierung der Anzahl von False Drops

- Geeignete Wahl der Entwurfparameter F , m und D
- Ziele:
 - Möglichst geringer Speicherplatz der Signaturen
 - Möglichst geringe Fehlerwahrscheinlichkeit für False Drops

⇒ Zielkonflikt

Fehlerwahrscheinlichkeit F_d

- Wahrscheinlichkeit, dass Signatur oder Blocksignatur Bedingung erfüllt, aber repräsentierte Dokumente das zu suchende Muster nicht enthält
- Formale Definition mittels Wahrscheinlichkeit:
 - $F_d = P(\text{Suchmuster nicht im Block enthalten} \wedge \text{Signatur erfüllt Bedingung})$
- Annahmen:
 - Anfrage besteht aus einem Wort
 - Gleichverteilung der Signaturen

Berechnung Fehlerwahrscheinlichkeit

- m/F : Wahrscheinlichkeit mit der ein Bit einer Wort-Signatur gesetzt ist
- $(1 - m/F)^D$: Wahrscheinlichkeit, dass Bit in Block-Signatur nicht gesetzt ist (Überlagerung wg. D)
- $1 - (1 - m/F)^D$: Wahrscheinlichkeit, dass Bit in Block-Signatur gesetzt ist
- $(1 - (1 - m/F)^D)^m$: Wahrscheinlichkeit, dass Blocksignatur zur Anfrage passt (m gesetzte Bits müssen stimmen; Kandidatenmenge)
- Wenn alle Kandidaten False Drops sind (sehr pessimistisch), dann $F_d = (1 - (1 - m/F)^D)^m$
- F steigt $\Rightarrow F_d$ fällt monoton
- D steigt $\Rightarrow F_d$ steigt monoton
- m steigt $\Rightarrow F_d$ kann fallen oder steigen
 $\Rightarrow$ Minimierung findet bestes m für vorgegebene Werte F und D

Optimierung für m

- für große D kann Formel für F_d (Binomialverteilung) durch Exponentialverteilung angenähert werden:

$$F_d = (1 - e^{-mD/F})^m$$

- daraus ergibt sich durch Ableitung und Umformung der optimale Wert für m als:

$$m_{\text{opt}} = F \cdot \ln 2 / D$$

- Einsetzen in die Formel zur Berechnung der Fehlerwahrscheinlichkeit:

$$F_d = (1/2)^{F \cdot \ln 2 / D}$$

- Verwendung von $m_{\text{opt}} \Rightarrow$ Anzahl der Einsen ist 50%

Praktische Anwendung der Fehlerwahrscheinlichkeit

- Speicherplatzberechnung:
 - Wort besteht im Durchschnitt aus c_w Zeichen
 - Zeichen benötigt 8 Bit
 - Y sei Gesamtzahl der Wörter in allen Dokumenten
 - Speicherplatzbedarf für Dokumente: $S_D = Y * c_w * 8$ Bit
 - Speicherplatzbedarf für Signaturen: $S_S = Y * F/D$ Bit
 - Wir setzen beide Werte in ein Verhältnis α : $S_S = \alpha * S_D$
 - Damit gilt: $F/D = \alpha * c_w * 8$

Praktische Anwendung der Fehlerwahrscheinlichkeit

- Beispiel $\alpha = 0,1$ und $c_w = 10$
 - $F/D = 8$, etwa $F = 80$ und $D = 10$
 - $F_d = 2,14\%$

- Beispiel $\alpha = 0,2$ und $c_w = 10$
 - $F/D = 16$, etwa $F = 80$ und $D = 5$
 - $F_d = 0,0459\%$

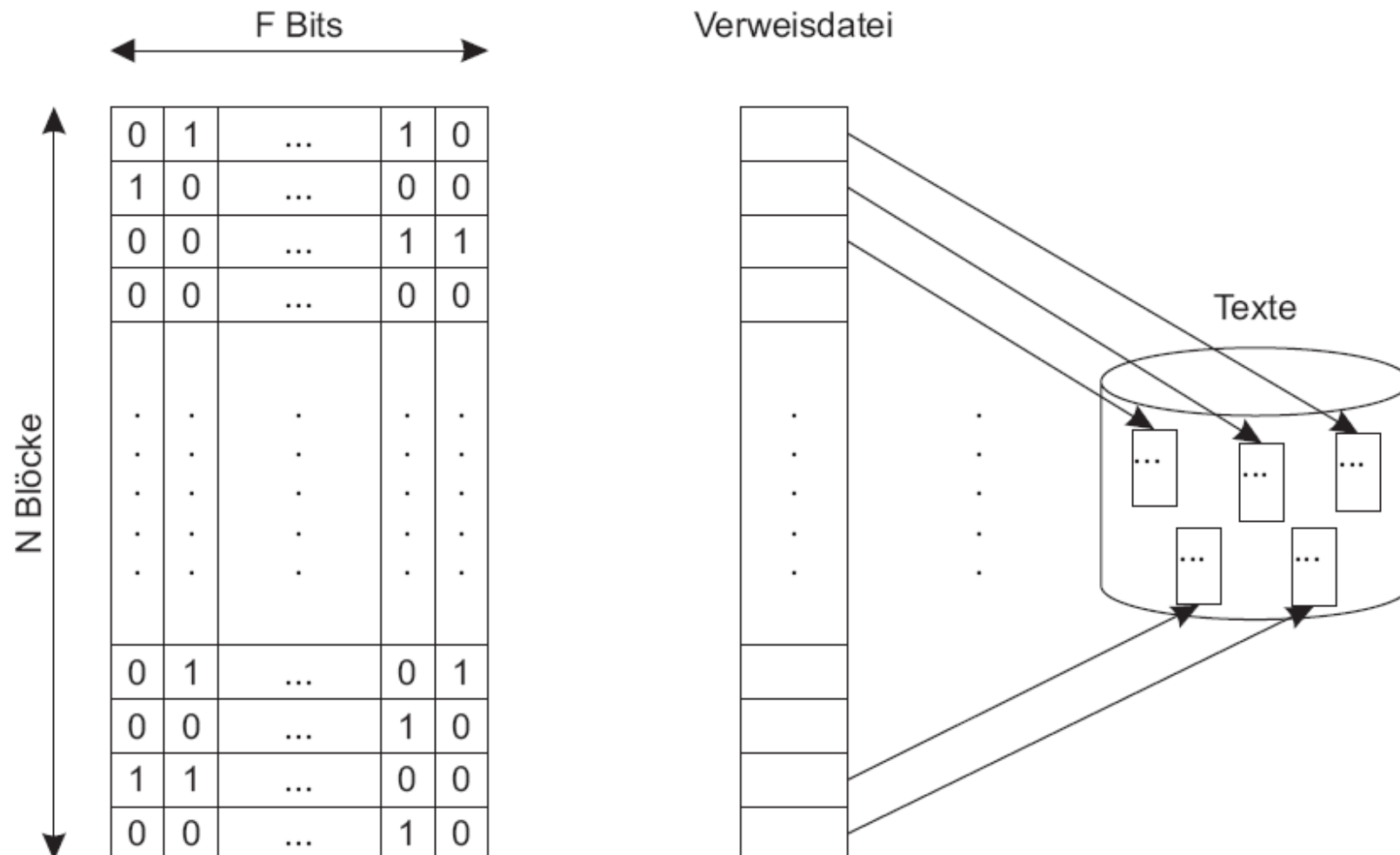
Speicherstrukturen für Signaturdateien

- drei Alternativen:
 - Sequentielle Signaturdateien
 - Bitscheibenorganisation
 - S-Baum

Sequentielle Signaturdateien

- Signaturdatei für N Blöcke ist Bitmatrix mit F Spalten und N Zeilen
 - Einfachste Speicherung: sequentielle Speicherung der einzelnen Zeilen der Matrix
⇒ SSF (Sequential Signature File)
 - Hinter jede Signatur Verweis auf Dokument
 - Alternative: alle Verweise in separate Datei sinnvoll, da nur einige Verweise gelesen werden müssen
-
- Suchaufwand wächst linear mit Größe Dokumentkollektion
 - u.U. passen Dateien in Hauptspeicher

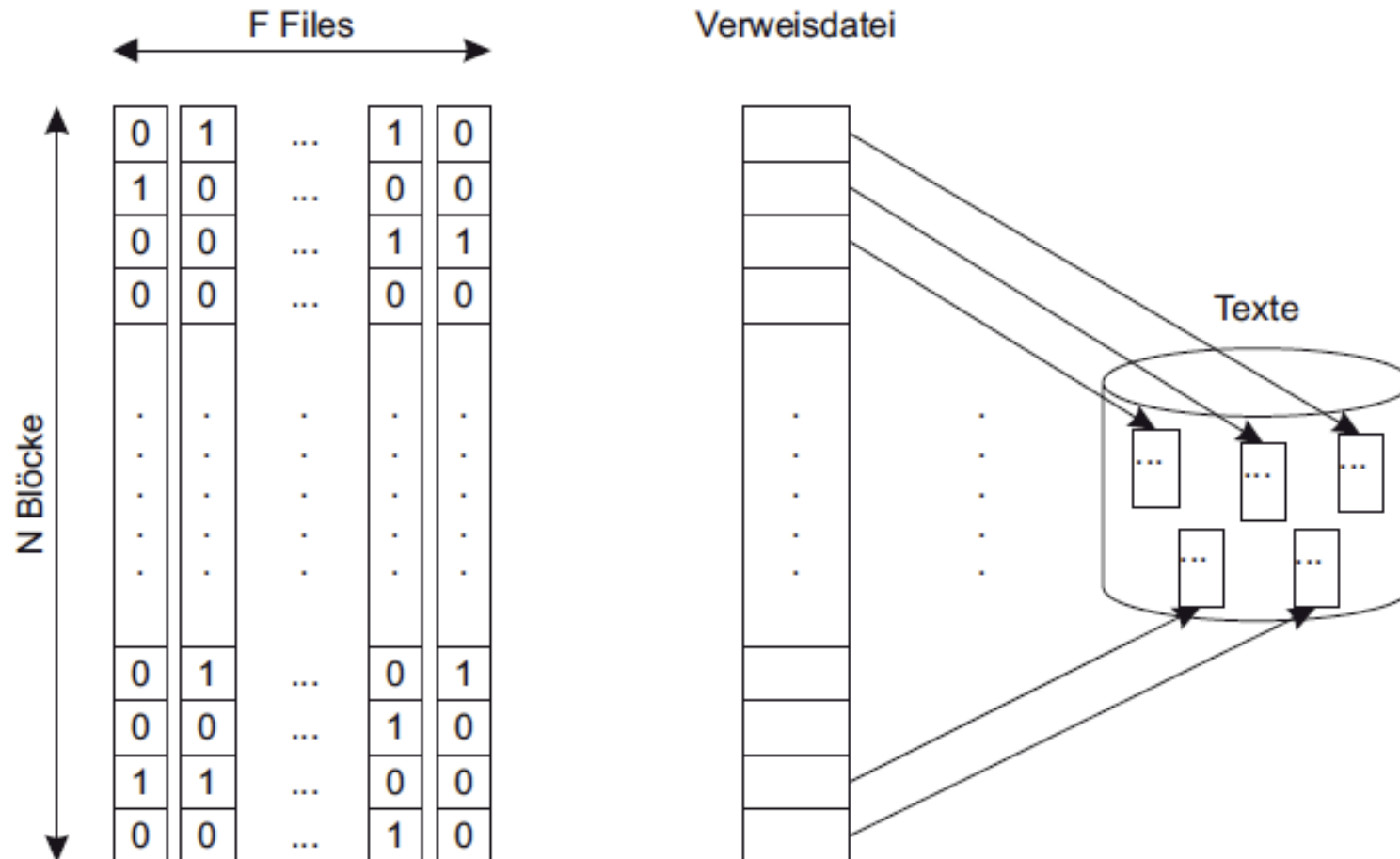
Sequentielle Signaturdateien



Bitscheibenorganisation

- SSF: Blocksignaturen werden komplett gelesen
- Eigentlich nur Spalten nötig, bei denen Anfragesignaturbits gesetzt sind
- Idee **BSSF** (Bit-Sliced Signature Files):
 - Jede Spalte in separate Datei $\Rightarrow$ F Dateien (Bitscheibe)
 - Pro Anfrage (ein Suchwort) müssen m Bitscheiben gelesen werden

Bitscheibenorganisation: Aufbau



Bitscheibenorganisation: Beispiel

Anfrage	Anfragesignatur
text search	010 010 101 100

Wort	Signatur
text	010 010 001 000
search	010 000 100 100
method	100 100 000 001
full	010 110 000 000
knowledge	000 111 000 000
retrieval	000 000 111 000
lexicon	100 010 010 000

Bitscheibenr.	2	5	7	9	10
<i>text search method</i>	1	1	1	1	1
<i>search knowledge retrieval</i>	1	1	1	1	1
<i>full text search</i>	1	1	1	1	1
<i>lexicon knowledge retrieval</i>	0	1	1	1	0

Block ist false drop

Block kein Kandidat

Bitscheibenorganisation: Bewertung

- Vorteil gegenüber SSF: nicht alle Bits müssen überprüft werden
- Nachteil: Änderungsoperationen benötigen immer F Festplattenzugriffe

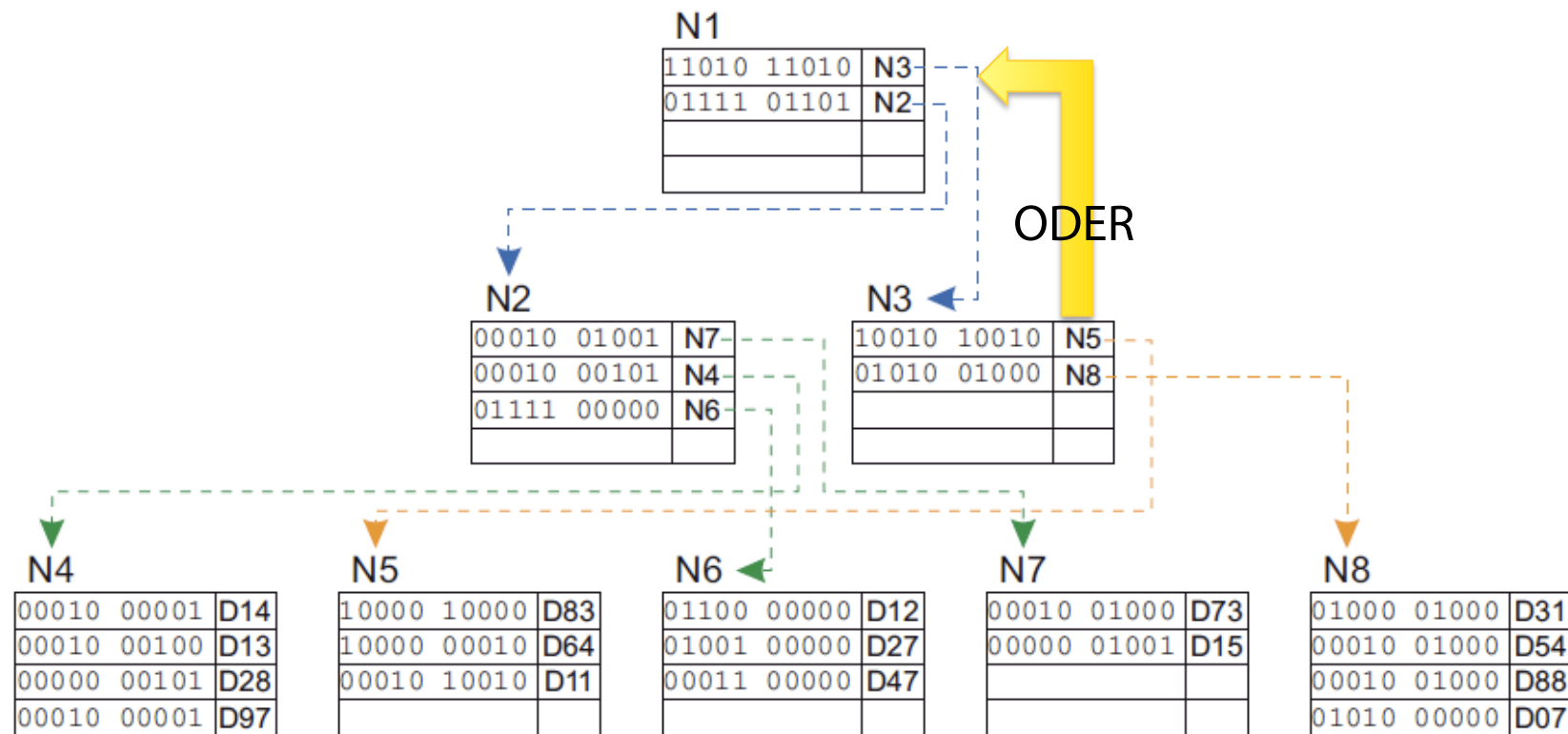
S-Baum

- Horizontale Aufteilung der $N \times F$ -Bitmatrix
- Verwandter des B-Baums
- Aufbau:
 - Jeder Knoten hat maximal k Einträge
 - Eintrag: Signatur und Adresse
 - Innere Knoten: Adresse verweist auf Kind im Baum
 - Blatt: Adresse verweist auf Dokument/Block

S-Baum: Überlagerung

- Signatur eines inneren Knotens ist ODER-Verknüpfung der Kind-Signaturen
- Problem: Knoten nahe der Wurzel können sehr viele Einsen enthalten

S-Baum: Aufbau



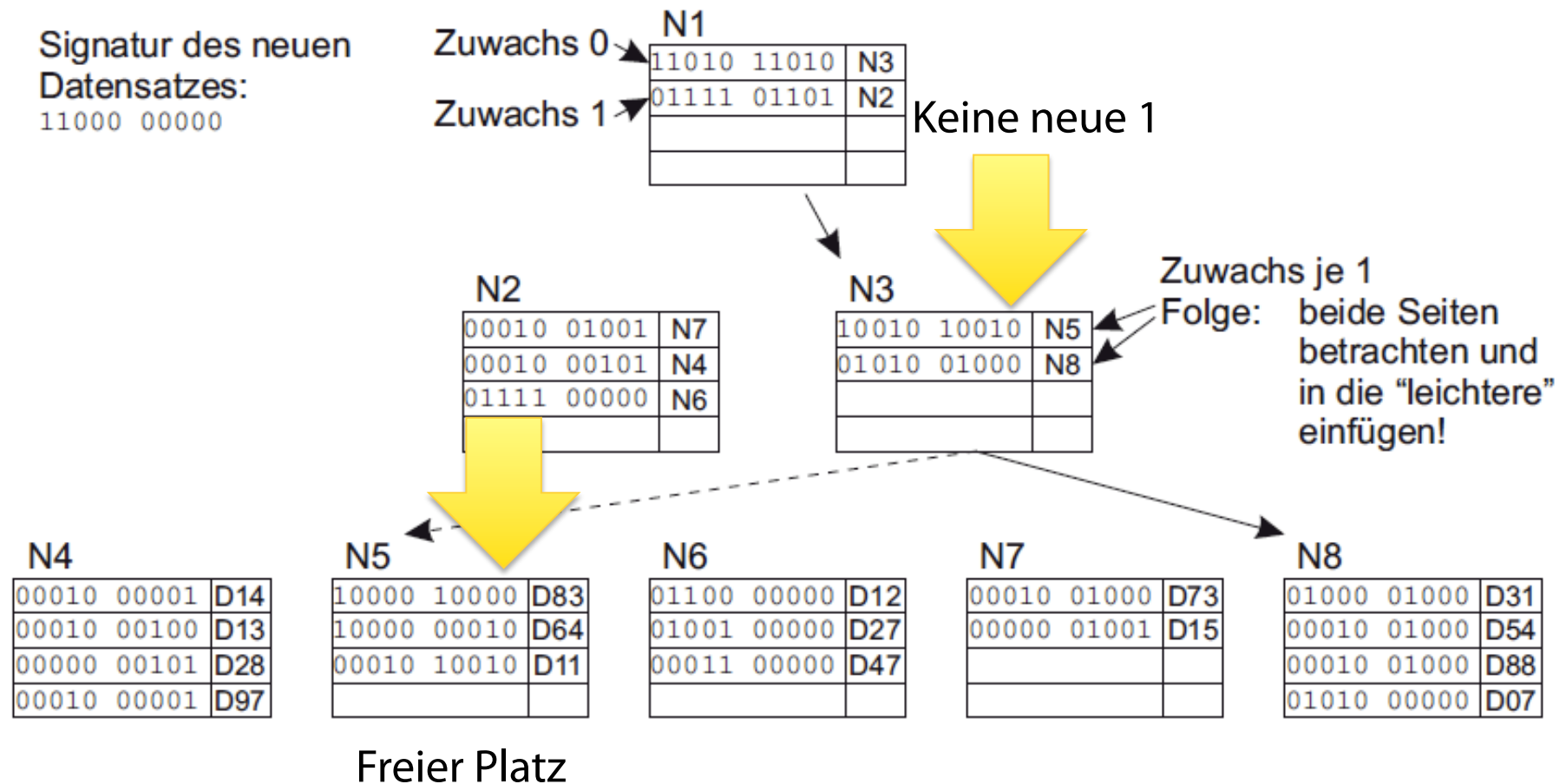
Suche im S-Baum

- Start bei der Wurzel
- Durchlauf aller Kinder, bei denen UND-Match mit Anfragesignatur erfolgreich
- Mehrere Suchpfade möglich
- Beispiel: Anfragesignatur 01000 00000 erzeugt Durchlauf von N1, N2, N3, N6, N8, D12, D27, D31, D07

Einfügen im S-Baum

- Navigieren zum Blatt über Knoten, die minimal erweitert werden müssten
- Überlauf:
 - Splitten des Blatts
 - Erzeugen zweier „Saatsignaturen“, welche maximal verschieden sein sollten (Aufwand $O(k^2)$)
 - Schnelle Heuristik zum Finden von Saatsignaturen: Suchen der Signaturen, deren Bits am weitesten links bzw. rechts liegen
 - Anschließend abwechselnd jeder Saatsignatur ähnlichste Signatur zuweisen

Einfügen im S-Baum



S-Baum: Fazit

- Vorteil: Suchaufwand wächst nur sublinear zur Größe der Dokumentkollektion
- Nachteil: Knoten nahe der Wurzel haben häufig sehr viele gesetzte Bits
⇒ wenige Knoten können von Suche ausgespart werden

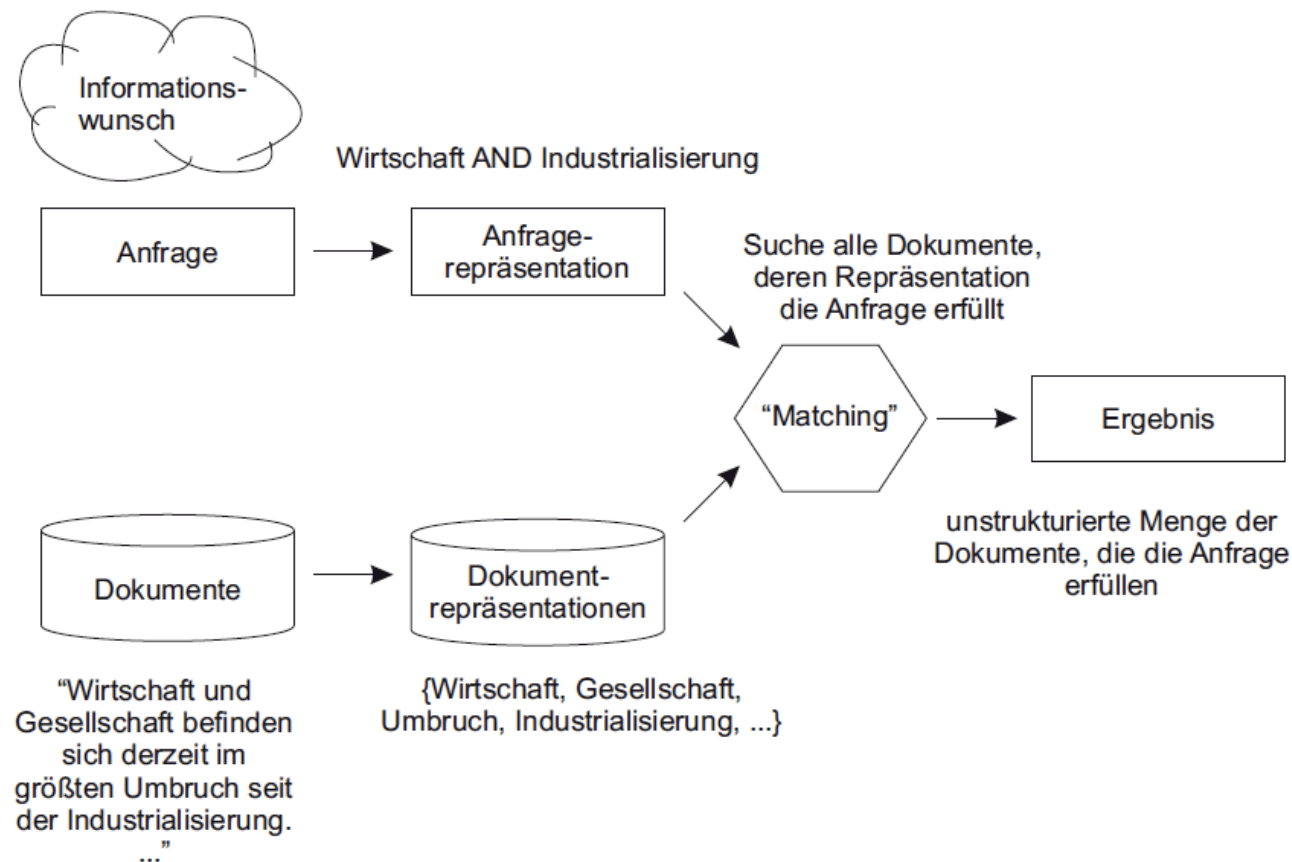
Vergleich der Signatur-Speicherstrukturen

Profil gibt Wichtigkeit von Operationen an

Struktur	Profil			
	Suche	Einfügen	Löschen	Speicher
sequentiell	selten	dominant	selten	wichtig
Bitscheiben	dominant	selten	selten	weniger wichtig
S-Baum	(wenig)	viel	viel	weniger wichtig

4.3 Boolesches Retrieval

■ Boolesche Ausdrücke über Suchwörtern



Dokumentenrepräsentation

- Dokument als Wortmenge (evtl. nach Stoppworteliminierung und/oder Stammformreduktion)
- Varianten:
 - **Multimenge** statt Menge
⇒ Speicherung der Vorkommenshäufigkeit
 - Menge von (Wort, Vorkommensort)-Paaren
 - Vorkommensort:
 - etwa im Titel, Überschrift oder Text
 - alternativ Wortposition im Text (Satznummer, Wortnummer)
⇒ sinnvoll für **NEAR**-Anfragen

Anfragerepräsentation

- Informationswunsch wird als Boolescher Ausdruck formuliert
- Wörter einer Wortmenge werden implizit ODER- oder UND-verknüpft
- Beispiele:
 - „Adler Bär“ $\Rightarrow$ „Adler“ UND „Bär“
 - „Adler AND (Bär OR Löwe)“
 - „Adler AND NOT Löwe“
 - BUT-Operator anstelle von AND NOT

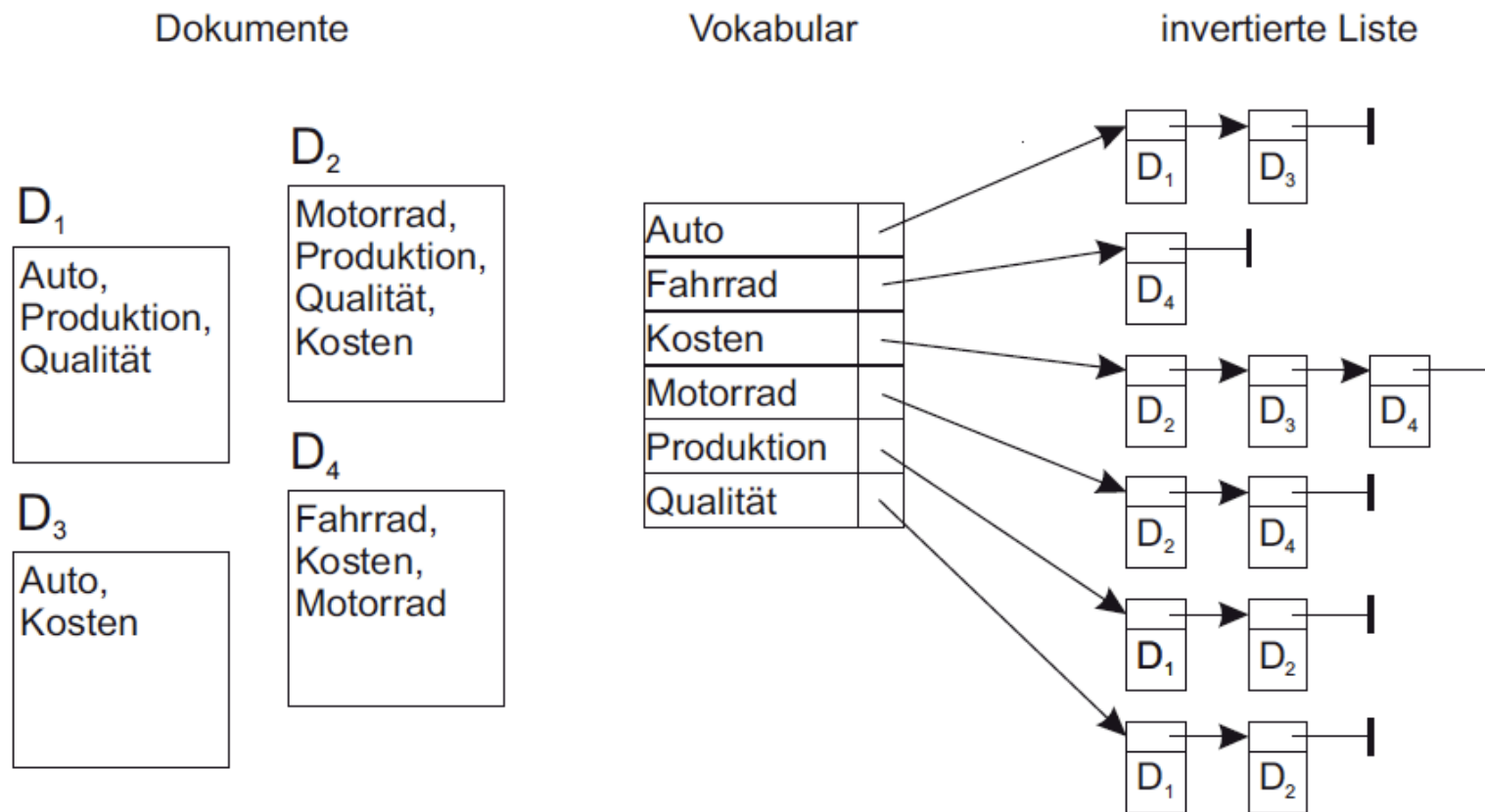
Implementierung

Varianten:

- Pattern Matching
sehr ineffizient bei größeren Dokumentkollektionen
- Signaturen
gut geeignet für UND-Verknüpfungen (aber nur blockweise)
- Invertierte Liste
vielseitigste Möglichkeit

Implementierung mit invertierten Listen

Einfache (ein Suchwort) Anfrage



Verknüpfte Anfragen

- UND-Verknüpfung
 - Schnittmengenberechnung der Listen
effizient unter Ausnutzung der Sortierung
- ODER-Verknüpfung
 - Vereinigung
- Unäre Negation $q = \text{NOT } q_2$
 - i.d.R. sehr großes Ergebnis, besser ist binäre Variante
- Binäre Negation $q = q_1 \text{ AND NOT } q_2$
 - Komplementärmenge

Weitere Operationen

- t_i NEAR[k] t_j
 - Term t_i darf nicht weiter als k Worte von Term t_j entfernt sein
 - Benötigt Vorkommensposition in invertierter Liste
 - Listen parallel durchlaufen und mögliche Paare testen
- IN_TITLE
 - Wort darf nur im Titel auftauchen

Boolesches Retrieval: Zusammenfassung

Probleme:

- In einfachster Form keine Reduzierung auf Grund- und Stammform
- Keine Gewichtung der Wörter nach Ort oder Häufigkeit möglich
- Keine Zerlegung von Mehrwortgruppen
- Kaum vorhersehbare Größe der Ergebnismenge
- Kein Ranking der Ergebnisse
- Anwender muss Boolesche Logik beherrschen

4.4 Coordination Level Match

Motivation:

- Beispielanfrage Boolesches Retrieval:
„Haus AND Garten AND Italien“
- Problem: Dokumente mit 2 Worttreffern erscheinen ebenso nicht wie Dokumente mit 0 oder 1 Treffer
- Ziel: wenn kein Dokument mit 3 Worttreffern
⇒ Dokumente mit 2 Treffern als Ergebnis

Coordination Level Match

Anfrage besteht aus

- Termen, die vorkommen sollen und
- Termen, die nicht vorkommen sollen

Beispiel: „Haus, Garten, NOT Frankreich“

Retrieval-Funktion addiert Werte auf (Ranking):

- Gewünschter Term im Dokument: +1
- Ungewünschter Term im Dokument: -1

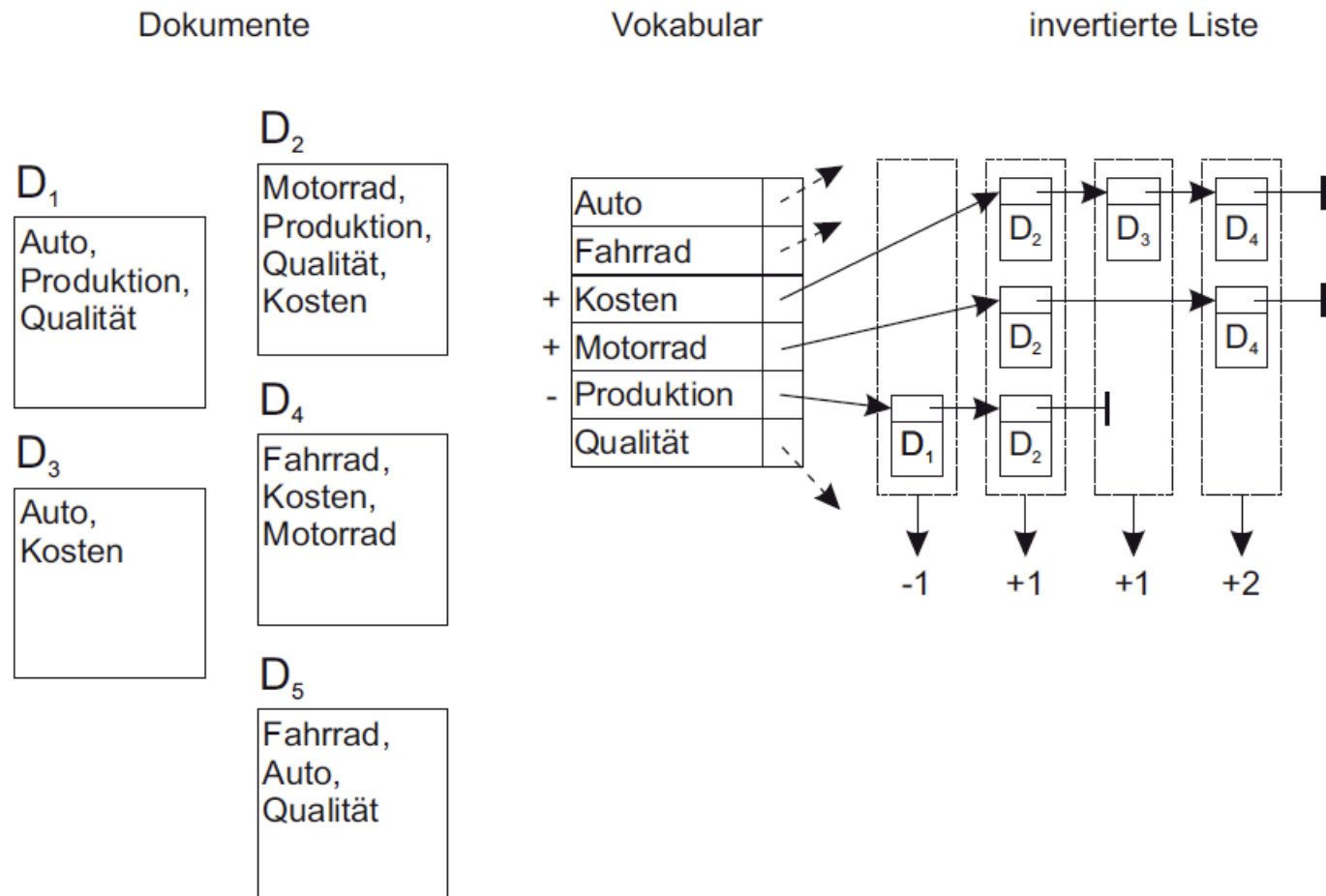
Beispieldokument: „Garten in Frankreich“ $\Rightarrow 0$

Implementierung

- Einfach mittels invertierter Listen
- Ausnutzung sortierter Listen
- Paralleler Durchlauf der Listen und Addition der Werte
- Ignorieren der Dokumente mit Werten kleiner oder gleich 0
- Anzeige der Ergebnisdokumente sortiert nach Wert
- Aufwand: $O(n*m)$ mit n für Dokumentanzahl und m für Anzahl zu durchlaufender Listen

Invertierte Listen

Anfrage: „Kosten, Motorrad, NOT Produktion“



Extended Boolean Retrieval

- Boolesches Retrieval führt zu niedriger Nutzerzufriedenheit
- „Aufweichung“ der strikten Mengensemantik durch Partial Matching
- Gewichtung des Vorkommens einzelner Terme und der Anfrageterme

Extended Boolean Retrieval

- z.B. Waller/Kraft (1979)

- Parametrisierter Junktor

$$t_1 \wedge / \vee \dots \wedge / \vee t_n$$

$$(1 - \gamma) * \min(w_{t_1}, \dots, w_{t_n}) + \gamma * \max(w_{t_1}, \dots, w_{t_n})$$

- Konjunktions-Charakteristik: $0 \leq \gamma \leq 0,5$

- Disjunktions-Charakteristik: $0,5 \leq \gamma \leq 1$

Extended Boolean mit Gewichten

- z.B. p-Norm-Modell (Salton et al.; 1983)

$$q_{or} = t_1 \vee^p \dots \vee^p t_n \quad q_{and} = t_1 \wedge^p \dots \wedge^p t_n$$

$$sim(q_{or}, d_j) = \left(\frac{\sum_{i=1}^n (x_i)^p}{n} \right)^{\frac{1}{p}}$$

$$sim(q_{and}, d_j) = 1 - \left(\frac{\sum_{i=1}^n (1 - x_i)^p}{n} \right)^{\frac{1}{p}}$$

$$1 \leq p \leq \infty$$

Eigenschaften p-Norm

■ $p=1$

$$\text{sim}(q_{or}, d_j) = \text{sim}(q_{and}, d_j) = \frac{\sum_{i=1}^n x_i}{n}$$

■ $p=\infty$

$$\begin{aligned}\text{sim}(q_{or}, d_j) &= \max(x_i) \\ \text{sim}(q_{and}, d_j) &= \min(x_i)\end{aligned}$$

4.5 Fuzzy-Mengen-Modell

- Idee: Anwendung Fuzzy-Mengen-Theorie (Zadeh) als Retrieval-Modell
- Boolesches Retrieval: Term in Dokument enthalten oder nicht
- Problem: Boolesche Werte sind zu scharf
- Beispiel: Dokument mit Term „SQL“ hat semantisch etwas zu tun mit Term „Datenbank“
- Ansatz: Zugehörigkeit Term zu Dokument als Zugehörigkeitswert aus $[0,1]$, wobei 1 bzw. 0 maximale bzw. minimale Zugehörigkeit bedeutet

Fuzzy-Menge

Ein Fuzzy-Menge S über einer Grundgesamtheit G wird durch Funktion $\mu_S : G \rightarrow [0,1]$ beschrieben, die für jedes Element aus G den Grad der Zugehörigkeit zu S angibt.

Verknüpfungen (von Zadeh vorgeschlagene Operationen):

- Negation: $\mu_{\neg S}(u) = 1 - \mu_S(u)$
- Disjunktion: $\mu_{A \cup B}(u) = \max(\mu_A(u), \mu_B(u))$
- Konjunktion: $\mu_{A \cap B}(u) = \min(\mu_A(u), \mu_B(u))$

Repräsentation der Anfrage/ Dokumente

- Anfrage analog zu Booleschem Retrieval
- Dokument D_j : $\mu_i(D_j)$ für alle Terme t_i des Vokabulars
- Retrieval-Funktion: Zadeh-Operationen

Repräsentation der Anfrage/ Dokumente: Beispiel

- Vokabular: t_1 =„Haus“, t_2 =„Garten“, t_3 =„Italien“, t_4 =„Frankreich“
- Anfrage: „Haus AND (Italien OR Frankreich) AND NOT GARTEN“
- Retrieval-Funktion: $\min(\min(\mu_1(D_j), \max(\mu_3(D_j), \mu_4(D_j))), 1 - \mu_2(D_j))$

Term	Dokumente D_j				
	D_1	D_2	D_3	D_4	D_5
Haus	0,5	0,0	0,7	0,3	0,9
Garten	0,9	1,0	0,1	0,0	0,0
Italien	1,0	0,0	0,0	1,0	0,0
Frankreich	1,0	1,0	0,4	1,0	0,5
Anfrage	0,1	0,0	0,4	0,3	0,5

Bestimmung der Zugehörigkeitswerte

Ansatz von Ogawa, Morita, Kobayashi [OMK91]

- Berechnung über **Term-zu-Term-Matrix** $[c_{i,l}]$
- $c_{i,l}$ ist Korrelation zwischen Termen t_i und t_l

$$c_{i,l} = \frac{n_{i,l}}{n_i + n_l - n_{i,l}}$$

- n_l ist Anzahl Dokumente, die t_l enthalten
- $n_{i,l}$ ist Anzahl Dokumente, die t_i und t_l enthalten
- Es gilt $c_{i,l}=1$ für $i=l$

Bestimmung der Zugehörigkeitswerte

Beispiel

- Term „Haus“ in 7 Dokumenten
- Term „Dach“ in 6 Dokumenten
- Beide zusammen in 5 Dokumenten
- $n_i = 7, n_l = 6, n_{i,l} = 5 \Rightarrow c_{i,l} = 5/8$

Bestimmung der Zugehörigkeitswerte

Berechnung Zugehörigkeit Term zu Dokument aus Term-zu-Term-Matrix

$$\mu_i(D_j) = 1 - \prod_{t_l \in D_j} (1 - c_{i,l})$$

- ersetze $1-x$ durch $\neg x$ und $x*y$ durch $x \wedge y$ und wende deMorgan und Regel der doppelten Negation an
 $\Rightarrow \mu_i(D_j) = \bigvee_{t_l \in D_j} c_{i,l}$
- Term-zu-Term-Werte des Dokuments werden faktisch **disjunktiv** verknüpft

Implementierung

Nutzung invertierter Listen

- Liste zu Term t_i enthält Dokumente D_j mit $\mu_i(D_j) > 0$
- Weiteres Problem: **lange** Listen
Abmildung mittels Grenzwert

Bewertung Fuzzy-Set Modell

■ Positiv

- Unterstützung der Vagheit der Sprache mittels Term-Korrelationen
- Ranking

■ Negativ

- Dominanz der Zadeh-Operationen *min/max*
- Komplexe Boolesche Anfrageformulierung
- Performance-Nachteile wegen langer Listen
- Term-zu-Term-Matrix auf dynamischer Dokumentkollektion (besser auf repräsentativem Datenbestand)
- Alternative: Term-Korrelation aus Thesaurus

Dominanzproblem der Zadeh-Operationen

- Anfrage: „*Haus AND Italien*“

Term	D_1	D_2
Haus	0,3	0,9
Italien	0,3	0,2
Anfrage	0,3	0,2

- System hält D_1 relevanter als D_2